

Analyzing Trajectory Gaps to Find Possible Rendezvous Region

ARUN SHARMA and SHASHI SHEKHAR, University of Minnesota, Twin Cities, USA

Given trajectory data with gaps, we investigate methods to identify possible rendezvous regions. The problem has societal applications such as improving maritime safety and regulatory enforcement. The challenges come from two aspects. First, gaps in trajectory data make it difficult to identify regions where moving objects may have rendezvoused for nefarious reasons. Hence, traditional linear or shortest path interpolation methods may not be able to detect such activities, since objects in a rendezvous may have traveled away from their usual routes to meet. Second, user detecting a rendezvous regions involve a large number of gaps and associated trajectories, making the task computationally very expensive. In preliminary work, we proposed a more effective way of handling gaps and provided examples to illustrate potential rendezvous regions. In this article, we are providing detailed experiments with both synthetic and real-world data. Experiments on synthetic data show that the accuracy improved by 50 percent, which is substantial as compared to the baseline approach. In this article, we propose a refined algorithm Temporal Selection Search for finding a potential rendezvous region and finding an optimal temporal range to improve computational efficiency. We also incorporate two novel spatial filters: (i) a Static Ellipse Intersection Filter and (ii) a Dynamic Circle Intersection Spatial Filter. Both the baseline and proposed approaches account for every possible rendezvous pattern. We provide a theoretical evaluation of the algorithms correctness and completeness along with a time complexity analysis. Experimental results on synthetic and real-world maritime trajectory data show that the proposed approach substantially improves the area pruning effectiveness and computation time over the baseline technique. We also performed experiments based on accuracy and precision on synthetic dataset on both proposed and baseline techniques.

CCS Concepts: • **Information Systems** → **Data Mining**; • **Computing Methodologies** → **Spatial and Physical Reasoning**;

Additional Key Words and Phrases: Spatial data mining, trajectory mining, time geography

ACM Reference format:

Arun Sharma and Shashi Shekhar. 2022. Analyzing Trajectory Gaps to Find Possible Rendezvous Region. *ACM Trans. Intell. Syst. Technol.* 13, 3, Article 36 (January 2022), 23 pages. <https://doi.org/10.1145/3467977>

This research is funded by an academic grant from the National Geospatial-Intelligence Agency (Award No. HM0476-20-1-0009, Project Title: Identifying Aberration Patterns in Multi-attribute Trajectory Data with Gaps). Approved for public release, 21-403.

Authors' address: A. Sharma and S. Shekhar, University of Minnesota, Twin Cities, Minneapolis, Minnesota, USA; emails: sharm485@umn.edu, shekhar@umn.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2157-6904/2022/01-ART36 \$15.00

<https://doi.org/10.1145/3467977>

1 INTRODUCTION

Moving objects such as ships often report trajectories with spatial and temporal gaps due to weak GPS signals, instrument malfunction, or malicious interference. Given multiple trajectories with spatial and temporal gaps due to such external factors, we find possible areas where moving objects (e.g., ships) may have rendezvous. Figure 1 shows a one-dimensional example of the problem Input and Output. The trajectory gaps of Objects 1 and 2 lie between P3 and P4 with maximum speed of 1 and 2, respectively. The output shows the **candidate active volume (CAV)** for each object. A CAV is the region in a gap that represents all the possible locations of the object during a missing time interval. The intersection of the two CAVs is a **spatiotemporal intersection (STI)** that represents a possible rendezvous region for the two objects. For a two-dimensional representation, please refer to Appendix A.

Societal applications for analyzing trajectory gaps are related to maritime safety, homeland security, epidemiology, and public safety. For instance, global security concerns such as transshipments, illegal oil transfer, illegal fishing, and so on, are important for maritime safety and regulatory enforcement. Such activities can be restricted and managed by identifying frequent missing signals from GPS trajectories of oil or fishing vessels. Other use cases from various domains are listed in Table 1. However, detecting potential rendezvous of objects such as ships is a time-intensive process, because it often requires manual inspection of gaps in trajectories spread over a large geographic space. Preliminary work done by human analysts can be substantially reduced with computational methods that narrows the search space to those regions where rendezvous could have occurred.

Traditional trajectory mining approaches use linear interpolation to characterize ship location during the gap and this can lead to many missed patterns. Figure 1 illustrates the missing patterns via traditional trajectory mining approaches. If we assume the missing segments of Objects 1 and 2 in the input are linear interpolations, then we are forced to conclude the objects never meet up. By contrast, our framework (shown in the output) is based on space-time prism models, which identifies a region where the objects could have potentially conducted a rendezvous. This results in missing patterns in real-world scenarios (e.g., illegal fishing, illegal cargo transfer, etc.). Another computational challenge is the large data volume distributed over considerable geographical space. For example, MarineCadastre [3], an **automatic identification system (AIS)** dataset, contains records with more than 30 attributes (e.g., location, draught) for 150,000 ships taken every minute during the years 2009 to 2017. Its total size is about 600 GB and covers all the waters around the U.S.

Many techniques implementing spatiotemporal joins (especially on trajectories) based on similarity measures and spatial indexes. For instance, the TS-Join [21] is applied in a spatial network and considers both spatial and temporal proximity while Time Relaxed Join [1] uses an effective technique to group objects moving in a similar pattern at different times. Other similarity joins [2, 7] perform time-interval threshold techniques such as sampling, sliding window, and so on. Query processing techniques based on 3D R-Trees have been proposed, such as Spatiotemporal R-Trees and **Trajectory-Bundle tree (TB-Tree)** [20]. However, the current literature has not addressed trajectory gap uncertainty or efficient pruning strategies inside the spatial join filter. Our research implements a Spatiotemporal Intersection Join of space time prisms with the addition of time-slicing operations for an effective spatial join filter pruning strategy (Figure 2).

In this article, we analyze the computational cost of determining potential rendezvous regions. The baseline method [22] uses a **Linear Temporal Scan (LTS)** Time-slicing algorithm inside a space-time grid and further reduces the geographical search space for rendezvous regions through additional time-slicing techniques. To improve the time complexity of Linear Temporal Scan, we

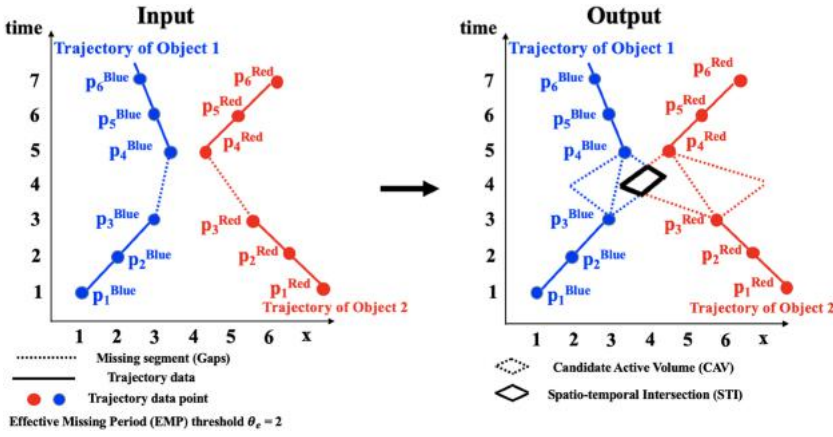


Fig. 1. An illustration of rendezvous region detection (best in color).

Table 1. Application Domain and Use Cases Examples for Detecting Rendezvous Regions

Application Domain	Example Use Cases
Astrophysics	Manual Tracking of astronomical objects (e.g., comets) that are no longer visible or temporarily lost in space.
Marine Biology	Tracking marine animals (e.g., whales, white sharks) trajectory movements in deep sea where signals tend to get lost for several minutes.
Contact Tracing	Identifying potential areas where people may have had contact with an infected person.
Mapping Coverage Gaps	Understanding potential areas of communication gaps or under capacity.

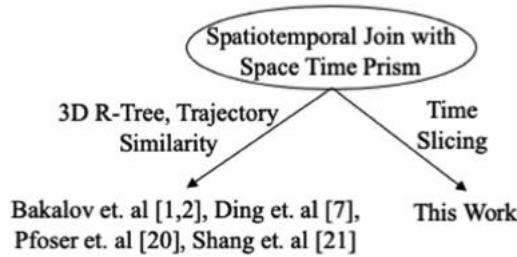


Fig. 2. Comparison of related work with proposed work.

propose a **Temporal Selection Search (TSS)**, which narrows down the temporal range closer to the actual intersection time range. Then, we apply a Static Ellipse Intersection filter and Dynamic Circle Intersection filter to further reduce the computational time. The proposed framework ensures completeness by finding all the possible rendezvous regions, such as (i) trajectories that actually met, (ii) trajectories that came close but did not meet, and (iii) trajectories that did not meet. While precision is low due to the large number of false positives (i.e., trajectories that came close but did not meet), recall is high, since we do not lose any information.

Contributions:

- Overall, this article investigates computational trajectory-gap handling techniques to improve accuracy (e.g., reduce false positives). Experiments on synthetic data show that the accuracy improved by 50 percent as compared to the baseline approach.
- We propose an efficient temporal pruning approach in TSS algorithm providing efficient temporal pruning data gaps modelled based on a space-time prism model. Experiments on real-world data show a significant improvement in computations over the baseline approach.
- We also propose spatial filters in a TSS algorithm: (i) Static Ellipse Intersection, (ii) Dynamic Circle Intersection. Static Ellipse intersection provides tighter filtering for more accurately detecting rendezvous and reduces spatial pruning as compared to the baseline approach. Dynamic Circle Intersection further reduces the filtering space for more efficient pruning during the time-slicing operation.
- We evaluate the proposed algorithms theoretically for Correctness, Completeness and Time Complexity of baseline and proposed algorithm.
- This article provides experimental evaluation of the proposed algorithm (TSS) based on area pruning effectiveness, computation time, accuracy and precision. The experiments are conducted while tuning different parameters such as Effective Missing Period and GPS readings.

Scope: In this work, we do not study kinetic prisms [14]. The proposed framework has multiple phases, but we limit this work to the filter phase. The refinement phase requires input from a human analyst and is not addressed here. Furthermore, the calibration of the cost model parameters is outside the scope of this work. In addition, we do not model rendezvous areas of trajectories without gaps, which are involved in intersections. Finally, we do not address the issue of positional accuracy while modeling the trajectories.

Organization: The article is organized as follows: Section 2 introduces the framework, basic concepts and the problem statement. Section 3 describes the baseline approach along with certain key concepts from our previous work [22]. We introduce the proposed Temporal Selection Search algorithm along with key concepts in Section 4. Section 5 provides theoretical proofs for correctness, completeness and time complexity. Experiment design and results are discussed in Section 6. Section 7 reviews the related work in more detail. Finally, Section 8 concludes the work and briefly lists future work.

2 POSSIBLE RENDEZVOUS DETECTION PROBLEM

2.1 Framework

Our aim is to identify possible rendezvous regions on a given set of trajectories through a two-phase *Filter* and *Refine* approach. Figure 3 shows a representation of the proposed framework. The trajectories are first preprocessed with a given **effective missing period (EMP)** threshold θ , whose output is a number of gap pairs greater than θ . These gap pairs then act as input to the framework as shown in Figure 3.

The framework includes two algorithms: a baseline Linear Temporal Scan [22], and the proposed TSS algorithm. Linear Temporal Scan is a basic time-slicing approach based on the union of two minimum orthogonal bounding rectangles. TSS expedites the appropriate time range search by calculating the start point and end point of the trajectories where an actual intersection may have occurred. This TSS algorithm further reduces the search space extent by introducing the intersection of two **minimum orthogonal bounding rectangles (MOBRs)** using 3D R-Tree and two **minimum bounding rectangles (MBRs)**. The output from the filter phase is then refined by finding the exact geometry of the cone intersect and accurately modeling the space time prism of

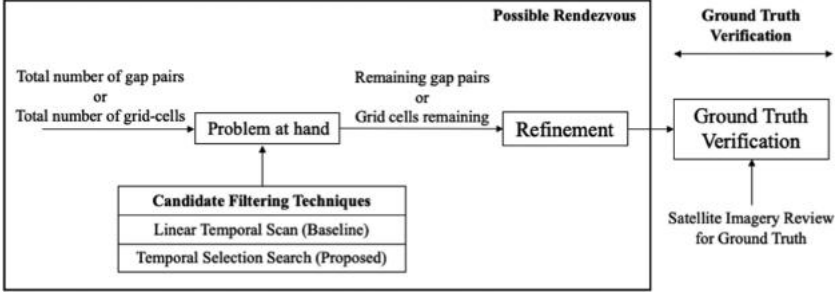


Fig. 3. Framework for detecting possible rendezvous regions to reduce manual inspection by analyst.

each object. The exact geometry can then be used by human analysts for ground-truth verification via satellite imagery.

2.2 Basic Concepts

In this section, several key concepts in the rendezvous detection problem are reviewed, which helps us to understand the problem formulation in Section 2.3.

Definition 2.1. A **study area** is a two-dimensional rectangular complying (latitude, longitude) coordinate system and represents the location of the given input data.

Definition 2.2. A **spatial trajectory** is a trace of chronologically sorted points in series generated by a moving object in a geographic space.

For example, $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n$ represents a spatial trajectory, $p_i = (x_i, y_i, t_i)$ where each point (p_i) is associated with a geographic coordinate set (x, y) and a time stamp (t).

Definition 2.3. An **object maximum speed** (S_{max}) is the maximum speed an object can attain based on the domain knowledge. The variable S_{max} can be identified from publicly available maritime vessel databases [3]. For vehicles, humans, or animals, we can use the maximum physically allowed speed.

Definition 2.4. An **effective missing period (EMP)** is a time period when a GPS signal is missing for longer than a user-specified EMP threshold (θ_e). Figure 1 shows that the EMPs for Object 1 and Object 2 are between timestamps 3 and 4 with $\theta_e = 2$.

Definition 2.5. A **candidate active volume (CAV)** is the spatiotemporal volume based on a space time prism, which provides the spatiotemporal bounds for all possible locations of an object during an EMP [9, 13, 15, 17, 18].

2.3 Problem Formulation

The problem to optimally identify rendezvous patterns in a spatio-temporal domain is formulated as follows:

Input:

- (1) A study area S ,
- (2) A set of $|N|$ trajectories $T = t_1 \dots t_{|N|}$, each associated with an object,
- (3) An object maximum speed (S_{max}) for each object,
- (4) An effective missing period threshold (θ_e).

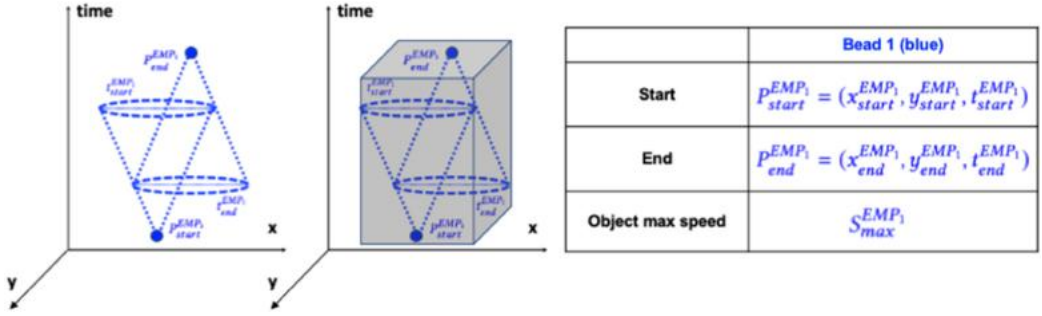


Fig. 4. Creating a minimum orthogonal bounding rectangle from two cones.

Output: Approximate geometry intersection of two gaps.

Constraints: (a) Trajectories have gaps, (b) maximum acceleration not available.

Objective: Improve **area pruning effectiveness (APE)** and computational efficiency.

Figure 1 shows a one-dimensional representation of two trajectory gaps involving a spatiotemporal intersection via overlap of two Candidate Avoidance Volumes.

APE is the ratio of the total study area and minimum bounded area inside the filtered region (i.e., approximate CAV):

$$\text{area pruning effectiveness (APE)} = \frac{\text{total study area}}{\text{area bounded inside the region}}, \quad (1)$$

where **area bounded inside the region** $\neq \emptyset$. Further, the higher the value of APE, the better the solution quality, since the minimum bounded area enclosed within a CAV will be smaller.

3 BASELINE APPROACH

Our previous work [22] was based on a time-slicing approach for effective pruning of pairs of gaps and improving area pruning efficiency. In this article, we take this work as a baseline and propose new spatial and temporal optimizations. The following summary of the baseline method lays the groundwork for our proposed new work that will be explained later in Section 4. Section 3.1 discuss MOBR creation and Spatiotemporal Intersection Criteria using space-time prism [17] and further approximation via Common MOBR. Extracting potential intersecting gaps (i.e., Candidate Pairs) via Spatiotemporal Intersection Criteria is discussed in Section 3.2 and the Linear Temporal Scan algorithm in Section 3.3.

3.1 MOBR Construction and Spatiotemporal Intersection Criteria

A MOBR acts as a base filter for the LTS Algorithm and the Spatiotemporal Intersection Criteria to check if two EMPs intersect. It is generated as follows:

Minimum Orthogonal Bounding Rectangle: Section 2.2 defined an EMP as the intersection of two cones (i.e., a bead) vertexed at the endpoints of a CAV. Therefore, to construct a new CAV, we first need to determine the coordinates of the end points attained via maximum speed and the radius attained during that time elapse. These parameters help us to find the endpoints to create a minimum orthogonal bounding rectangle over a given CAV. For instance, in Figure 4, we construct a CAV from point $p_{end}^{EMP_1}$ at any time instance t based on radius r , i.e., from the product of $S_{max}^{EMP_1}$ and time elapsed $(t_{end}^{EMP_1} - t)$. A similar operation is performed at $p_{start}^{EMP_1}$ at any time instance with radius $r_{start}^{EMP_1} = (t - t_{start}^{EMP_1}) \times S_{max}^{EMP_1}$. To construct a MOBR, we define the radius at time $t = t_{end}^{EMP_1}$ from $p_{start}^{EMP_1}$ or at $t = t_{start}^{EMP_1}$ from $p_{end}^{EMP_1}$. Then, we find the endpoints

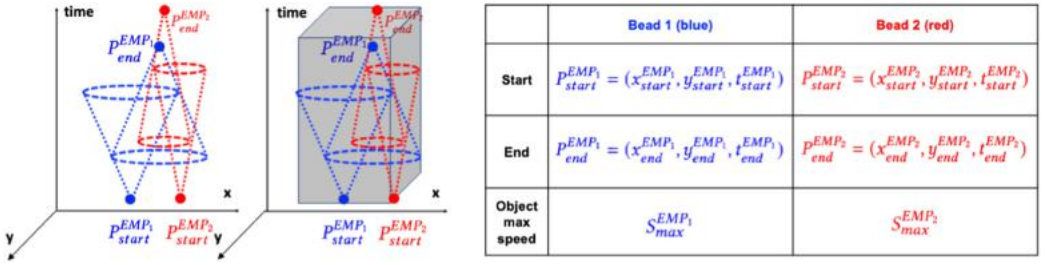


Fig. 5. Cone intersection of two effective missing period (EMPs).

by calculating the relative radial shift in coordinates from the start and end points. This operation is done for both CAVs. Finally, we take the overall maximum and minimum coordinates from the two CAV's respective endpoints and create a common MOBR (Figure 5).

Spatiotemporal Intersection Criteria: To compute the intersection between two EMPs, we need to determine the intersections between the four cones (i.e., beads) generated from the endpoints of the EMPs. We first check whether any two beads have an overlapping time range. If none do, then the two EMPs are guaranteed to be not intersecting. To be intersecting, there must be at least one timestamp when sections of the two beads intersect, i.e., if two beads have an overlapping time range, at least one-time stamp (start-point or end point) of one of the gap segments must be within the time range of the other. To illustrate this, we take two data gaps, EMP_1 and EMP_2 having time range $t_{start}^{EMP_1}, t_{end}^{EMP_1}$ and $t_{start}^{EMP_2}, t_{end}^{EMP_2}$, respectively, and check whether the difference between $(t_{start}^{EMP_1}, t_{end}^{EMP_2})$ or $(t_{end}^{EMP_1}, t_{start}^{EMP_2}) \geq 0$. If it is, then the two EMPs satisfy the necessary condition of a two EMP intersect.

For the sufficient condition, we use the radius information of two cones from a different EMP and check whether their respective radii will overlap with each other. For two circles to overlap, the sum of their radii must be greater than the distance between their respective centers. For instance, the sum of the radius of the cone from start point $r_{start}^{EMP_1}$ of EMP_1 and end point $r_{end}^{EMP_2}$ of EMP_2 must be greater than the distance between their respective radii centers $d(start, end)$. Equation (2) provides the sufficient condition for two EMPs to be considered intersecting:

$$r_{start} + r_{end} \geq d(start, end), \quad (2)$$

where $start = \{p_{start}^{EMP_1}, p_{start}^{EMP_2}\}$, $end = \{p_{end}^{EMP_1}, p_{end}^{EMP_2}\}$, and $d(start, end)$ is the Euclidean distance between points start and end points. Here, we consider the radius generated at the endpoints of the EMPs for MOBR construction.

3.2 Extracting Potential Intersecting Gap Pairs

In our preliminary work [22], the extraction of gaps was based on a filter and refine approach where a given set of gaps are first sorted to produce potential candidate pairs CP . The idea is to first reduce I/O storage cost in the filter phase and minimizing the computation in the refinement phase. The sufficient and necessary condition for the spatiotemporal intersection of two CAVs and the creation of a minimum orthogonal bounding rectangle is explained in Section 3. The final output is then saved in a candidate pair list that will further refined in the time-slicing operation for the Linear Temporal Scan algorithm.

3.3 Linear Temporal Scan Algorithm

The LTS is based on the idea of a 3D filtering technique that leverages spatiotemporal properties and additional attributes of the ST prism model for a better geometric approximation of a bounded

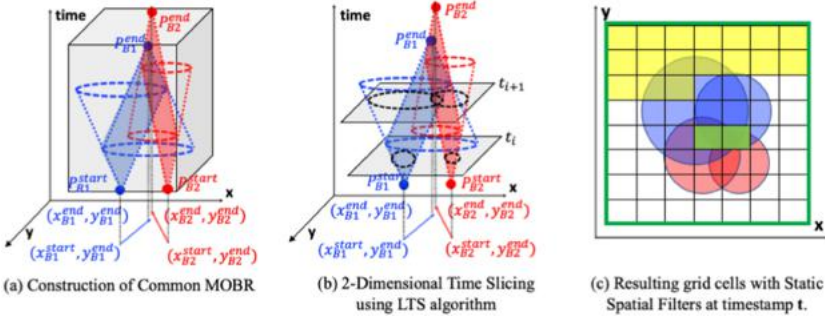


Fig. 6. Linear temporal scan (LTS) algorithm.



Fig. 7. Extracting common time interval list.

region. This results in a more optimal filter as compared to a minimum orthogonal bounded rectangle area that is relatively larger than the actual intersection area.

LTS starts by extracting $|N|$ potential candidate pairs and saving them in a candidate pair list. Next, it creates a spatiotemporal grid with a specified spatial and temporal threshold. A common temporal range is selected and candidate pairs with a common MOBR are approximated as described in Section 3.1. Then, we perform time-slicing operations by generating circular disks D from the common start point to the common end point from both EMPs and check if any grid cell $grid(x, y)$ resides in a given circular disk. If it does, then the grid cell $grid(x, y)$ is saved in an output list. The pseudo-code of LTS algorithm is given in Algorithm 1.

Step 1: Extract Potential Candidate Pairs (MOBRs): First, we extract potential candidate pairs where two CAVs prove to be intersecting as per the necessary and sufficient conditions discussed in Section 3.1. The candidate pairs are then saved in a candidate pair list ranging from CP_1 to CP_n using the methodology mentioned in Section 3.2. More details related to gap extraction are given in our previous work [22].

Step 2: Create a Spatiotemporal Grid: Second, we create a spatiotemporal grid with the size of the given study area and a spatial and temporal resolution m and n , respectively.

Step 3: Extract Time Interval List: We formalize a common time range as $[t_{start}^{STI}, t_{end}^{STI}]$ by $[\max(t_{start}^{EMP1}, t_{start}^{EMP2}), \min(t_{end}^{EMP2}, t_{end}^{EMP1})]$ as illustrated in Figure 7. This operation is followed by indexing of the time interval points and populating the temporal units within $t_{start}^{STI}, t_{end}^{STI}$ in a Time Slice List.

Step 4: Perform Time Slicing: Before the time-slicing operation, a spatial approximation is performed over each EMP of a candidate pair (since each CP_i is the union of two EMPs) based on a MOBR using maximum speed and time elapsed from within $[t_{start}^{STI}, t_{end}^{STI}]$ (shown in Figure 6(a)). Then, spatial grid cells $grid(x, y)$ are saved in a Static Spatial Grid. Finally, a time-slicing operation is performed for each temporal unit defined by temporal resolution m and start point t_{start}^{STI} and end point t_{end}^{STI} . For each time slice (or temporal unit), we construct Circular Disks $D_{start}^{EMP1}, D_{end}^{EMP1}, D_{start}^{EMP2}$, and D_{end}^{EMP2} whose centers are the terminal points of the EMPs and radius as the product of maximum speed $S_{max}^{EMP1}, S_{max}^{EMP2}$ and time elapsed (as discussed in Section 3.1). Then, for each

$grid(x, y)$ inside a predefined Static Grid Filter, we check whether the given $grid(x, y)$ resides in the intersection of the dynamically constructed Circular Disk (i.e., $D_{start}^{EMP_1} \cap D_{end}^{EMP_1} \cap D_{start}^{EMP_2} \cap D_{end}^{EMP_2}$). If it does, then the given $grid(x, y)$ is saved in a *TimeSliceGridList*, which is constructed at each time slice operation. The *TimeSliceGridList* is also saved in a *LocalGridList* for each candidate pair. This list is eventually saved in a *GlobalGridList* representing all the possible rendezvous locations of the given study area.

ALGORITHM 1: Linear Temporal Scan (LTS) Algorithm

Input:

- A Study Area S
- A set of $|N|$ Gap Pairs with given EMPs and Maximum Speed
- Spatial Resolution $\leftarrow m$
- Temporal Resolution $\leftarrow n$

Output:

- *GlobalGridList* $\in$ Total Intersection Area of Candidate Pair List

1: procedure :

2: Initialize *GlobalGridList* $\leftarrow \emptyset$

3: Step 1: Extract Potential Candidate Pairs

4: Candidate Pair List $\leftarrow [CP_1, CP_2, \dots, CP_n]$

5: Step 2: Create Spatiotemporal Grid

6: Spatial grid $\leftarrow \{grid(0, 0), \dots, grid(m, m)\}$

7: Temporal grid $\leftarrow \{grid(x, y, 0), \dots, grid(x, y, n)\}$

8: for each: $CP_i \in$ Candidate Pair List do

9: Initialize *LocalGridList* $\leftarrow \emptyset$

10: $CP_i \in \{EMP_1 \cup EMP_2\}$

11: Step 3: Extract Time Interval List

12: $[t_{start}^{STI}, t_{end}^{STI}] = [\max(t_{start}^{EMP_1}, t_{start}^{EMP_2}), \min(t_{end}^{EMP_1}, t_{end}^{EMP_2})]$

13: Time Slice List $\rightarrow [t_{start}^{STI}, t_{end}^{STI}]$

14: Step 4: Perform Time Slicing

15: Static Filter $\leftarrow$ Common MOBR Filter ($EMP_1, EMP_2, S_{max}^{EMP_1}, S_{max}^{EMP_2}$)

16: Static Spatial Grid $\leftarrow grid(x, y) \in$ Static Filter

17: for each: Time Slice $\in$ Time Slice List do

18: Initialize *TimeSliceGridList* _{i} $\leftarrow \emptyset$

19: Construct Circular Discs $D_{start}^{EMP_1}, D_{start}^{EMP_2}, D_{end}^{EMP_1}, D_{end}^{EMP_2}$

20: for each: $grid(x, y)$ in Static Spatial Grid do

21: if $grid(x, y) \in [D_{start}^{EMP_1} \cap D_{end}^{EMP_1} \cap D_{start}^{EMP_2} \cap D_{end}^{EMP_2}]$ then

22: *TimeSliceGridList* _{i} $\leftarrow grid(x, y)$

23: *LocalGridList* _{i} $\leftarrow$ *TimeSliceGridList* _{i}

24: *GlobalGridList* $\leftarrow$ *LocalGridList* _{i}

Execution trace of LTS Algorithm: Figure 8 shows the execution trace of Step 4 of Algorithm 1 in a two-dimensional representation with the intersection of two CAVs over a 12×10 grid. The x-axis represents longitude and the y-axis, time. Prior to time slicing, we first decide the common temporal time range $[t_{start}^{STI}, t_{end}^{STI}]$ where *STI* stands for Spatiotemporal Intersection and $t_{start}^{STI}, t_{end}^{STI}$ are the actual times of the intersection from the start and end points followed by

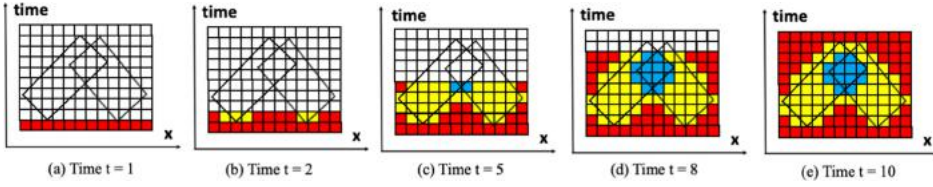


Fig. 8. Execution trace of linear temporal scan (LTS) algorithm.

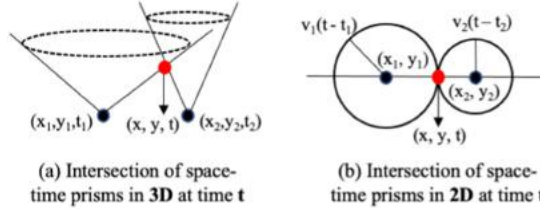


Fig. 9. Intersection of two space-time prisms.

a common MOBR approximation. Grid cells $grid(x, y)$ residing in none of the CAVs are shown in red. Cells residing in one of the CAVs are shown in yellow while cells that reside in both CAVs are colored blue. As shown in Figure 8(a), no grid cell $grid(x, y)$ resides in any of the CAVs at time $t = 1$, whereas in a subsequent iteration, some cells do reside in one of the CAVs at $t = 2$ (shown in Figure 8(b)). Figures 8(c) and 8(d) show the common grid cells inside both CAVs in blue from time $t = 5$ to $t = 8$. These common grid cells serve as the intersection of four circular discs D_{start}^{EMP1} , D_{start}^{EMP1} , D_{start}^{EMP2} , and D_{start}^{EMP2} in three-dimensional space as shown in Figure 6(c), where the green cells are the common intersection of the four circular discs (i.e., blue cells in Figures 8(c), 8(d), and 8(e)).

4 PROPOSED APPROACH

LTS provides fine pruning to help a human analyst to scan minimal approximate areas for refinement. However, the baseline LTS technique proves to be inefficient in terms of computational time when we increase spatial and temporal resolution. In our new work, we propose a TSS, a temporal optimization method based on the concept of a space-time prism intersection [12], which helps us to get prior temporal information on when two space time prisms actually met. Section 4.1 explains the basic intuition and concept for finding a smaller actual time range as compared to the baseline approach. In addition, spatial optimizations using 3D R-Trees and minimum bounding rectangle intersection along with Dynamic Circle Intersection are also proposed. Section 4.2 describes spatial optimization across each temporal unit based on (i) a Static Ellipse Intersection Filter and (ii) a Dynamic Circle Intersection Filter, which significantly reduces the spatial bounds, resulting in tighter pruning. Finally Section 4.3 formalizes the Time Prioritize Approach and proposed spatial optimization.

4.1 Time Prioritized-based Temporal Selection Pruning

The time interval of a Linear Temporal Scan from t_{start}^{STI} to t_{end}^{STI} is based on a common time selection as discussed in Section 3.3. However, selecting a common time may not be efficient in terms of overall computational time especially when the temporal range in t_{start}^{STI} to t_{end}^{STI} is not significant. Therefore, we incorporate the concept of finding a temporal range using space-time prism intersection [12] in Step 3 of Algorithm 1 when selecting the appropriate range of t_{start}^{STI} to t_{end}^{STI} .

For instance, in Figure 9(a), (x_1, y_1, t_1) and (x_2, y_2, t_2) are the start points of two space-time prisms whose intersection can be imagined as the two-dimensional intersection of two circles at point

(x, y, t) (as shown in Figure 9(b)). However, in a realistic scenario, the actual intersection point or initial contact (x, y, t) happens much later than either prisms start points (x_1, y_1, t_1) and (x_2, y_2, t_2) . Hence, prior knowledge of the time interval when a prism actually intersect can also contribute toward saving multiple time-slicing operation resulting in early termination.

Section 3.1 briefly summarized the sufficient condition for a Spatiotemporal Intersection by comparing the distance of two points with their respective individual radii using the equation:

$$r_{start} + r_{end} \geq d(start, end), \quad (3)$$

where $start = \{P_{start}^{EMP_1}, P_{start}^{EMP_2}\}$, $end = \{P_{end}^{EMP_1}, P_{end}^{EMP_2}\}$, and $d(start, end)$ is the Euclidean distance between points start and end points.

To satisfy this condition, we use the radius information of two cones from a different gap and check whether their respective radii will overlap with each other. Two circles are considered overlapping if the sum of their radii is smaller than the distance between their respective centers. For finding the initial contact, we check whether the two circles touch each other (e.g., Figure 9) by checking if the sum of their radii is **equal** to the distance between their respective centers. This motivates a search for a new start time t_{start}^{STI} and end time t_{end}^{STI} interval by deriving the terms and solving Equation (3):

$$t \geq \frac{d(start, end) + t_{start} \times S_{max}^{EMP_1} + t_{end} \times S_{max}^{EMP_2}}{S_{max}^{EMP_1} + S_{max}^{EMP_2}} \rightarrow \text{where } start = P_{start}^{EMP_1}, end = P_{start}^{EMP_2}, \quad (4)$$

$$t \leq \frac{-d(start, end) + t_{start} \times S_{max}^{EMP_1} + t_{end} \times S_{max}^{EMP_2}}{S_{max}^{EMP_1} + S_{max}^{EMP_2}} \rightarrow \text{where } start = P_{end}^{EMP_1}, end = P_{end}^{EMP_2}, \quad (5)$$

$$\frac{d(start, end) + t_{start} \times S_{max}^{EMP_1} + t_{end} \times S_{max}^{EMP_2}}{S_{max}^{EMP_1} + S_{max}^{EMP_2}} \leq t \leq \frac{-d(start, end) + t_{start} \times S_{max}^{EMP_1} + t_{end} \times S_{max}^{EMP_2}}{S_{max}^{EMP_1} + S_{max}^{EMP_2}}, \quad (6)$$

$$t_{start}^{STI} \leq t \leq t_{end}^{STI}. \quad (7)$$

Hence, if the two space-time prisms are found to intersect, then t_{start}^{STI} and t_{end}^{STI} acts as a new time interval in Step 3 of Algorithm 1 when two space time prisms actually intersect.

4.2 Spatial Filter for Temporal Selection Search Algorithm

The spatial approximation of space-time prisms via a common MOBR was proposed in the baseline approach can be further optimized by leveraging the geometric properties of the prisms. The main intuition is to minimize the $grid(x, y)$ comparison with $[D_{start}^{EMP_1} \cap D_{end}^{EMP_1} \cap D_{start}^{EMP_2} \cap D_{end}^{EMP_2}]$ in Step 4 of the Algorithm 1. Hence, we propose two spatial optimization techniques, (i) Static Static Ellipse Intersection and (ii) Dynamic Circle Intersection.

Static Ellipse Intersection: Given the start and end points of the EMP, a space-time prism can be projected onto an x - y spatial grid [17] via an ellipse, which spatially delimits the extent of a moving object's mobility with maximum speed v_{max} . In Figure 10(a), start point (x_1, y_1, t_1) and end point (x_2, y_2, t_2) with $t_1 < t_2$ are bordered by an ellipse with foci (x_1, y_1) and (x_2, y_2) . The equation for the ellipse represented in Figure 10 can be stated as

$$\sqrt{(x - x_1)^2 + (y - y_1)^2} + \sqrt{(x - x_2)^2 + (y - y_2)^2} \leq (t_2 - t_1) \times v_{max}, \quad (8)$$

where (x_1, y_1) and (x_2, y_2) are the foci and (x, y) is any given point on the ellipse. The major axis a is the product of the total distance travelled with given time interval $t_2 - t_1$ and maximum

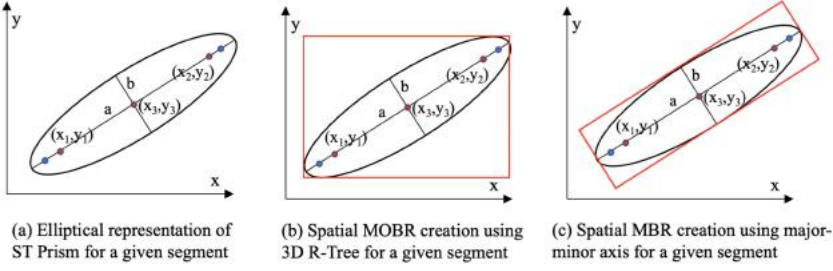


Fig. 10. Different spatial approximation filter around static ellipses.

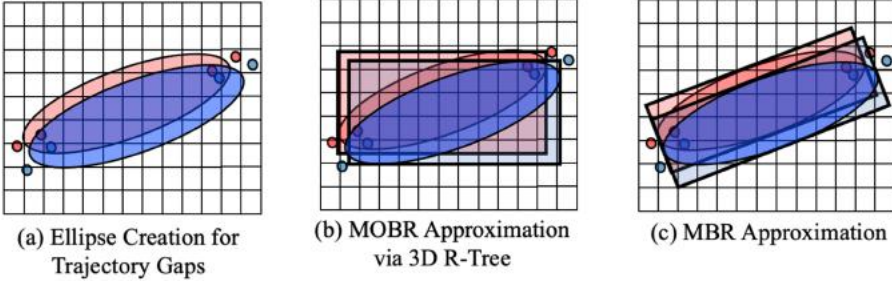


Fig. 11. Ellipse intersection via space-time prisms.

speed v_{max} divided by 2 (i.e., $a = \frac{v_{max} \times (t_2 - t_1)}{2}$). Minor axis length b can be derived by subtracting $v_{max}(t_2 - t_1)$ with the Euclidean distance from (x_1, y_1) and (x_2, y_2) . Hence, defining a basic spatial approximation such as a MOBR or MBR around ellipses proves to give us a much tighter bound, resulting in fewer cells as compared to the common MOBR used for the baseline in Section 3.1.

Figure 10(b) shows an approximating ellipse with a minimum orthogonal bounding rectangle via a 3D R-Tree. The approximation is done by calculating the end points of the major axis using midpoint (x_3, y_3) , a and slope $m_1 = \frac{(y_2 - y_1)}{(x_2 - x_1)}$. Using major axis endpoints as diagonals and time elapsed, we generate a 3D Minimum Orthogonal Bounding Rectangle with the help of 3D R-Trees. Figure 10(c) shows an approximating ellipse with a MBR. The MBR can be constructed using midpoint (x_3, y_3) , the end points of the major axis and the perpendicular slope to the major axis, which was calculated while constructing the MOBR (i.e., $-\frac{1}{m_1}$). This provides even better spatial approximation as compared to 3D R-Trees intersection with an even fewer number of grid cell comparisons.

For a rendezvous of two or more objects, calculating the intersection of such a spatial approximation (e.g., Figure 11(a)) is more computationally efficient than performing an approximation based on a common MOBR (as described in Section 3.1). For instance, the intersection of the two ellipses shown in Figure 11(b) shows that the intersection of two MOBRs approximated via the intersection of ellipses using a 3D R-Tree provides better approximation as compared to a common MOBR. Further in Figure 11(c), the intersection of the newly approximated MBR of the two ellipses provides an even better bound, resulting in even fewer grid cells as compared to 3D R-Trees and Baseline. This helps us to not only identify the rendezvous regions held of the base filter with a fewer number of $grid(x, y)$ cells but also provides more accurate rendezvous regions than the MOBR, which will further reduce false positives.

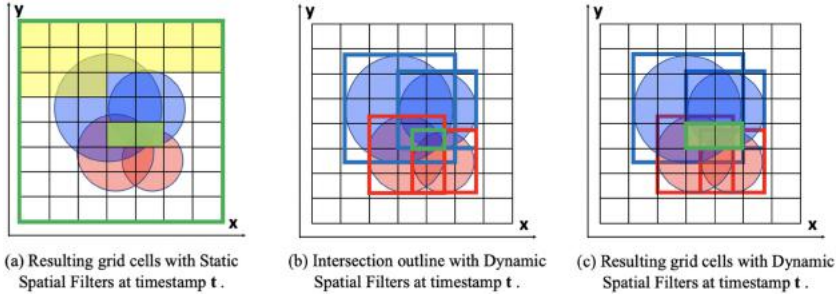


Fig. 12. Different spatial approximations for dynamic circular intersection.

Dynamic Circle Intersection: As noted in Section 3, given start point (x_1, y_1, t_1) and end point (x_2, y_2, t_2) with speed v , the cones at each temporal unit t ($t_i < t < t_{i+1}$) are represented as two-dimensional circles. Such information provides dynamic spatial bounds throughout the temporal life-cycle of a space time prism. Hence, the uncertainty of a space-time prism of a given CAV can be summarized by three polynomial inequalities [17]:

$$t_i \leq t \leq t_{i+1}, \quad (9)$$

$$(x - x_i)^2 + (y - y_i)^2 \leq (t - t_i)^2 v^2, \quad (10)$$

$$(x - x_{i+1})^2 + (y - y_{i+1})^2 \leq (t_{i+1} - t)^2 v^2, \quad (11)$$

where Inequality (9) defines the time range defined by the effective missing period *EMP* of a CAV and Inequality (10) and Inequality (11) provide circle equations within the ST Prism from the start point and end point, respectively. In the case of finding a Rendezvous area, we perform a spatiotemporal intersection join of two space time prisms as discussed in Section 3. Hence, each grid cell needs to qualify under Inequalities (10) and (11) for both space time prisms for the rendezvous area. However, the baseline approach proves expensive, since creation of a common MOBR depends on the maximum radius generated from the endpoints for both CAVs for a large number of $grid(x, y)$ comparisons. This motivates us to reduce the spatial bounds at each time stamp based on the intersection of four circles (i.e., Inequalities (10) and (11) of both ST Prisms), resulting in a significant decrease in the number of comparisons.

For instance, Figure 12 shows a comparison operation to determine whether each grid cell $grid(x, y)$ resides in all four circles at a given time stamp t . Figure 12(a) shows each cell $grid(x, y)$ (yellow) and the actual grid cells residing in all four circles (green). Figures 12(b) and 12(c) show intermediate operation of spatial approximation by Dynamic Circle Intersection where Figure 12(b) shows the intersection of MOBRs (approximated around the each circle) resulting in smaller MOBR, which is a significantly smaller rectangular area than given by the LTS time-slicing operation. The MOBR bounds are then indexed on a static filter (e.g., 3D R-Tree or Ellipse intersection). This results in a shorter comparison operation when the resulting intersection rectangle is indexed to the nearest grid cells as shown in Figure 12(c).

4.3 Temporal Selection Search Algorithm

TSS extends the idea of the LTS by leveraging the spatiotemporal and geometric properties of space-time prism models for optimal geometric approximation. Temporal Selection Search starts with a similar approach as the baseline, i.e., finding potential intersection candidates and constructing a space-time grid. A temporal range is selected using the time prioritizer approach

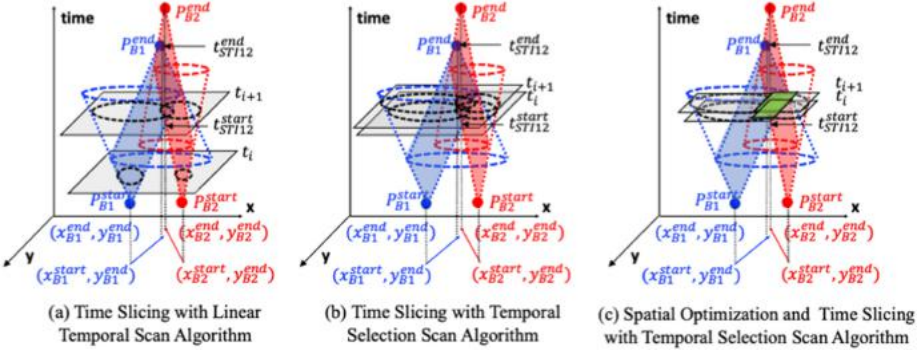


Fig. 13. Comparison of temporal selection search algorithm with baseline approach.

(discussed in Section 4.1) to minimize the number of linear temporal scan operations. Finally, a spatial optimization method such as Dynamic Circle Intersection Filtering (discussed in Section 4.2) is performed to further minimize the number of $grid(x, y)$ comparisons, resulting in a lower time cost.

Figure 13 shows a comparison of the proposed Temporal Selection Search algorithm with the baseline Linear Temporal Scan approach. In the baseline approach, the time-slicing operation is done on timestamps in time interval range $[\max(t_{start}^{EMP1}, t_{start}^{EMP2}), \min(t_{end}^{EMP1}, t_{end}^{EMP2})]$ involving time-slicing operations (e.g., Figure 13(a)) where no actual intersection of circular discs occurs. Temporal Selection Search provides a better pruning strategy by using a Time Prioritizer (Equations (4) and (5)) to get $t_{start}^{STI}, t_{end}^{STI}$, significantly reducing the time-slicing operation as shown in Figure 13(b). In addition, spatial filtering is also applied using the Static Elliptical and Dynamic Circle Intersection Filters providing a significantly tighter filter than baseline technique (shown in Figure 13(c)). Algorithm 2 shows the pseudo-code for TSS.

Steps 1 and 2 are similar to Algorithm 1.

Step 3: Extract Time Interval List: In this step, the Time Prioritizer extracts a relatively smaller time interval range, i.e., from t_{start}^{EMP1} to t_{end}^{EMP2} . Then the time interval points are indexed via a linear search $[t_{start}^{STI}, t_{end}^{STI}]$ and enumerated in the Time Slice List.

Step 4: Perform Time Slicing: This final step is similar to the baseline approach except for some extra operations related to spatial approximation and intersection. In Step 11, we approximate the proposed space-time prism with static elliptical bounds with a MBR by leveraging geometric properties of the ellipses. This involves construction of tighter bound ellipse with physics-based parameters, e.g., speed, time elapsed for each missing segment in the pair and will result in a smaller search space, i.e., fewer grid cell comparisons (discussed in Section 4.2). The proposed static ellipse filter then serves as the base filter for the time-slicing operation. During time slicing, we construct four disks for each time slice and check that their intersection (i.e., $D_{start}^{EMP1} \cap D_{end}^{EMP1} \cap D_{start}^{EMP2} \cap D_{end}^{EMP2}$) is not empty. If it is not, then we construct a spatial approximation such as a bounding box around each disk and perform an intersection to get Intersection MOBR Bounds. These bounds are then indexed in the base filter and the resulting grid cells are saved as a Dynamic Spatial Grid variable. Finally, we check if any $grid(x, y)$ resides in the intersection of the original four discs and qualifying grid cells are saved inside $TimeSliceGridList_i$, which is further saved in $LocalGridList$ for each candidate pair. A $GlobalGridList$ then saves the $LocalGridList$, representing the rendezvous regions of the entire study area.

ALGORITHM 2: Temporal Selection Search Algorithm (TSS)**Input:**

- A Study Area S
- A set of $|N|$ Gap Pairs with given EMPs and Maximum Speed
- Spatial Resolution $\leftarrow m$
- Temporal Resolution $\leftarrow n$

Output:

- GlobalGridList $\in$ Intersection Area of $CPairList$

```

1: procedure :
2:   Initialize GlobalGridList  $\leftarrow \emptyset$ 
3:   Step 1 and 2 are same as Algorithm 1
4:   for each:  $CP_i \in CPairList$  do
5:     Initialize LocalGridList $i$   $\leftarrow \emptyset$ 
6:      $CP_i \in \{EMP_1 \cup EMP_2\}$ 
7:     Step 3: Extract Time Interval List
8:      $[t_{start}^{STI}, t_{end}^{STI}] = \text{Time Prioritizer}(EMP_1, EMP_2, S_{max}^{EMP_1}, S_{max}^{EMP_2})$ 
9:     Time Slice List  $\rightarrow [t_{start}^{STI}, t_{end}^{STI}]$ 
10:    Step 4: Perform Time Slicing
11:    Static Filter  $\leftarrow$  Optimized Spatial Filter ( $EMP_1, EMP_2, S_{max}^{EMP_1}, S_{max}^{EMP_2}$ )
12:    Static Spatial Grid  $\leftarrow grid(x, y) \in$  Static Filter
13:    for each: Time Slice  $\in$  Time Slice List do
14:      Initialize TimeSliceGridList $i$   $\leftarrow \emptyset$ 
15:      Construct Circular Discs  $D_{start}^{EMP_1}, D_{end}^{EMP_1}, D_{start}^{EMP_2}, D_{end}^{EMP_2}$ 
16:      if  $[D_{start}^{EMP_1} \cap D_{end}^{EMP_1} \cap D_{start}^{EMP_2} \cap D_{end}^{EMP_2}] \neq \emptyset$  then
17:        Disk MOBR List  $\leftarrow$  SpatialApproximation( $D_{start}^{EMP_1}, D_{end}^{EMP_1}, D_{start}^{EMP_2}, D_{end}^{EMP_2}$ )
18:        Intersection MOBR Bounds  $\leftarrow$  MOBRIntersection(Disk MOBR List)
19:        Index Intersection MOBR Bounds with Static Spatial Grid
20:        Dynamic Spatial Grid  $\leftarrow grid(x, y) \in$  Indexed Intersection MOBR
21:        for each:  $grid(x, y)$  in Dynamic Spatial Grid do
22:          if  $grid(x, y) \in [D_{start}^{EMP_1} \cap D_{end}^{EMP_1} \cap D_{start}^{EMP_2} \cap D_{end}^{EMP_2}]$  then
23:            TimeSliceGridList $i$   $\leftarrow grid(x, y)$ 
24:          LocalGridList $i$   $\leftarrow$  TimeSliceGridList $i$ 
25:    GlobalGridList  $\leftarrow$  LocalGridList $i$ 

```

Execution trace of Temporal Selection Search Algorithm: Figure 14 shows the execution trace of Step 4 for the Algorithm 2 representation with the intersection of two CAVs over a 12×10 grid where the x-axis represents longitude and the y-axis, time. Before proceeding to time slicing, a time range $[t_{start}^{STI}, t_{end}^{STI}]$ is decided by the Time Prioritizer algorithm, providing a smaller estimation as compared to baseline. In the figure, grid cells residing in only one of the CAVs are colored yellow while cells residing in both CAVs are blue. The time-slicing operation starts at time $t = 5$ (as shown in Figure 14(a)) with the static spatial filter and a common circle intersection area (yellow) in any of the CAVs. Figure 14(b) shows the circle intersection filter providing tighter bounds compared to the static filter, resulting in filtering minimal number of cells. A similar operation is shown in Figures 14(c) and 14(d) for time $t = 8$, where the time-slicing operation terminates.

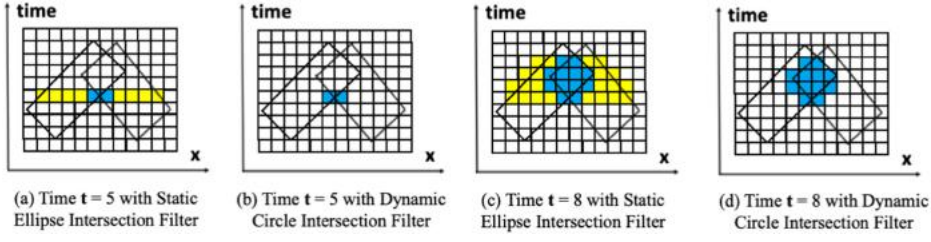


Fig. 14. Execution trace of temporal selection search algorithm.

5 THEORETICAL EVALUATION

In this section, we present a theoretical evaluation of the proposed TSS algorithm and show that it is complete but not always correct due to the large presence of false positives in the results. We also provide a brief time complexity analysis for extracting gaps in Section 5.2 and time-slicing operation in Appendix B.

5.1 Correctness and Completeness

LEMMA 5.1. *Given a finite set of trajectories, Temporal Selection Search terminates in finite time.*

PROOF. Given a finite set of $|K|$ trajectories, each trajectory may have a maximum N number of finite points. Therefore, we have a maximum bound of $|K| \times |N|$ finite trajectory points to process for finite candidate pair generation. Hence, for a finite number of trajectories and gap pairs, a finite number of operations will be performed for the Temporal Selection Search algorithm. $\square$

LEMMA 5.2. *Temporal Selection Search is complete.*

PROOF. A search algorithm is considered complete if it returns all true positive cases. This may include some cases with false positives but no false negatives. The temporal bounds of TSS are based on Effective Missing Periods of both objects is the subset of the time interval range $[t_{start}^{STI}, t_{end}^{STI}]$ generated at Step 12 of LTS algorithm. Further, the spatial bounds of a rendezvous region at Step 18 will be a subset of the proposed base-filter (MBR) for each time slice from $[t_{start}^{STI}, t_{end}^{STI}]$. Hence, TSS will consider all possible rendezvous cases are follows:

- (i) When a rendezvous actually occurs: TSS return all true positives (i.e., identifies positive rendezvous when actual rendezvous take place). This includes cases when objects appear to cross paths or remain very close to each other, generating a guaranteed intersection of spatial filters, which results in a rendezvous region being returned.
- (ii) When objects try to approach but do not meet: TSS may return false positives (i.e., identifies positive rendezvous when no actual rendezvous take place). This may occur when objects are so close, they appear to meet, which may result in the intersection of spatial filters and subsequent generation of a rendezvous region.
- (iii) When objects do not meet: TSS may return false positives or true negative (i.e., identify a negative rendezvous when no actual rendezvous take place). This may occur when objects are far apart or have smaller overlapping time range, since, in certain cases, spatial filters will do interact, generating a rendezvous region.

In all three cases, rendezvous regions are created on top of spatial filters, which act as a superset of the maximum extent of the spatial bounds at each time slice of an actual rendezvous region. This also leads to a large number of false positives, which may result in low precision or accuracy

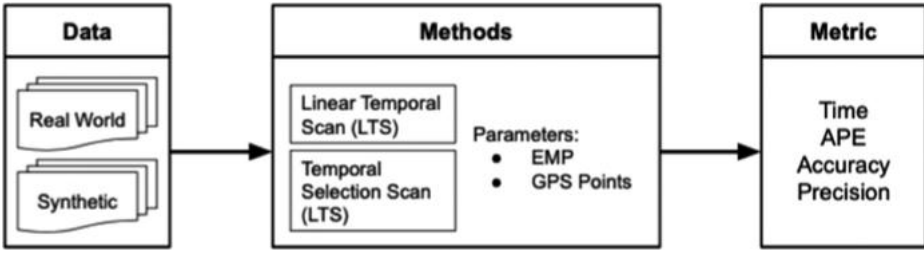


Fig. 15. Experiment design.

but due to no false positive cases, recall will remain perfect. Hence TSS is complete but not always correct. $\square$

5.2 Asymptotic Time Complexity Analysis

Exacting Candidate Gap Pairs: Given $|K|$ unique trajectories each associated with $|N|$ trajectory points, there are $|K| \times |N - 1|$ total number of gap pairs. For generating candidate pairs, $\binom{|K|}{2}$ trajectories must be selected for potential candidate pairs where each such pair will have $|N - 1| \times |N - 1|$ comparisons. Hence, the total number of comparisons will be $\binom{|K|}{2} \times |N - 1| \times |N - 1|$, which result in asymptotic worst case of $O(|K|^2 \times |N|^2)$.

6 EXPERIMENTAL EVALUATION

In this section, we evaluated the proposed Temporal Selection Search algorithm by comparing it against the baseline Linear Temporal Scan. Section 6.1 provides brief overview of Experiment Design along with Comparative Study, Sensitivity Analysis, and dataset description. Section 6.2 shows comparison of baseline and proposed method over each performance metrics.

6.1 Experiment Design

The goal is to evaluate and compare the proposed Temporal Selection Search approach against the baseline Linear Temporal Scan under different parameters. The performance metrics were APE, computation time, accuracy and precision. Figure 15 shows detailed Experiment Design.

Comparative Study: We compared TSS and LTS via experiments under fixed set of parameters such as Effective Missing Period and number of GPS points. The experiments are based on performance metrics such as APE, Computation Time, Accuracy and Precision. Then, we varied these parameters to perform sensitivity analysis where TSS clearly outperforms the baseline LTS.

Sensitivity Analysis: The parameters were chosen to answer two questions: For a given study area, we asked:

- (1) What is the effect of the EMP threshold?
- (2) What is the effect of the number of GPS Points?

Synthetic Dataset: We created a synthetic dataset using a set of real-world trajectories from MarineCadastre [3] and considered those trajectory pairs that spatially cross each other. Then, we removed overlapping trajectory points to create a hypothetical rendezvous empty space and temporally synchronized the trajectory pairs to extract a time interval list as discussed in Section 3.3. The experiments were done on over the same number of trajectories with manual labelling for ground-truth verification. The manual labelling was based on actual rendezvous for ground-truth comparison.

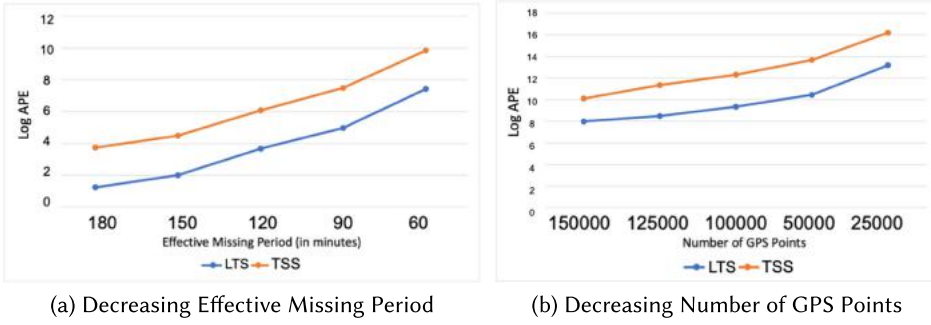


Fig. 16. Effect of varying parameters on area pruning effectiveness.

Real-world Dataset: We used real-world data from the MarineCadastre [3] dataset. The dataset contains many attributes (e.g., Longitude, Latitude, **Speed Over Ground (SOG)**, etc.) for 150,000 objects from 2009 to 2017. The dataset has a geographical extent of 180W to 66W degrees in longitude and 90S to 90N degrees in latitude (based on the WGS 1984 coordinates system). In experiments, we used the attributes Longitude, Latitude, Time, SOG, and **Maritime Mobile Service Identity (MMSI)**. Maximum speed was calculated by taking the average SOG from both endpoints of the EMP.

Computing Resources: All the experiments were conducted using Python and performed on an Intel Core i5 2.5GHz CPU and 16GB memory.

6.2 Experiment Results

In this section, we perform a comparative study and sensitivity analysis over performance metrics such as Area Pruning Effectiveness and Computation Time, Accuracy, and Precision. In sensitivity analysis, we vary parameters such as Effective Missing Period from 60 to 180 min and number of GPS points from 2.5×10^4 to 1.5×10^5 with EMP at 60 min.

(1) Area Pruning Effectiveness: APE is the ratio of the total study area and minimum bounded area inside the filtered region. The formulae is presented in Section 2.3. This indicates that if APE is higher, the enclosed minimum bounded area will be lower resulting in better solution quality. The main intuition of APE is to reduce the refinement cost by reducing computation time of each cell per unit area [22]. The comparison is based on LTS and TSS using spatial filters 3D R-Trees and MBR, respectively, where ellipse bounded MBR provides tighter bounding area as compared to 3D R-Trees. The spatial approximations of 3D R-Trees and ellipse bounded MBR is performed at Step 15 of Algorithm 1 and Step 11 of Algorithm 2, respectively. The details of the spatial approximation steps for TSS can be found in Step 4 (Perform Time Slicing) in Section 4.3.

Trends: The results of the experiments are shown in Figure 16. As can be seen, in all cases, TSS achieves higher area pruning effectiveness than LTS. The results also reveal how APE is affected by different parameters. Figures 16(a) and 16(b) shows a reverse relationship between APE and the other two parameters, i.e., APE increases with decreasing EMP and number of GPS points. This means the shorter the Effective Missing Period, the less likely an overlap will occur, resulting in a smaller average rendezvous area and higher Area Pruning Effectiveness. Similarly, the larger the number of GPS Points, the greater the number of potential gap pairs, resulting in a larger average rendezvous area and smaller Area Pruning Effectiveness.

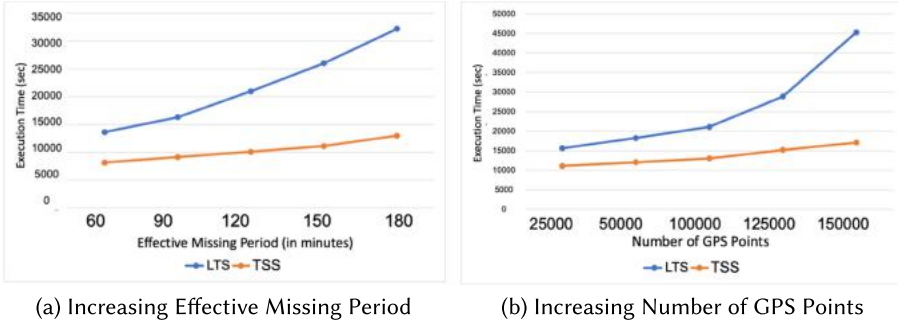


Fig. 17. Effect of varying parameters on computation time.

Actual Class	Predicted Class		
		Positive	Negative
	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Fig. 18. Evaluation confusion matrix.

(2) Computation Time: Computation Time is the total time taken by the baseline LTS and proposed TSS algorithms. Our main purpose is to provide an efficient algorithm in terms of computation time in filter phase. The unit for time measurement is in seconds.

Trends: Using the same set values and parameter increases as the APE experiments, we compared the run times for the baseline LTS and proposed TSS methods. The results are shown in Figure 17. As can be seen, increasing the Effective Missing Period (Figure 17(a)), and number of GPS points (Figure 17(b)) leads to longer computation times. This is because longer EMP increases the overlap between candidate pairs, and a greater number of GPS points means more candidate pairs to analyze. Nevertheless, in both cases, the proposed Temporal Selection Search method outperforms the baseline LTS.

We also performed evaluations based on accuracy and precision for ground-truth verification. The experiments are based on evaluating correctness of the baseline and proposed algorithm for detecting rendezvous using spatial interaction of their respective spatial approximation filters (i.e., 3D R-Tree and MBR of LTS and TSS, respectively). The outcomes for both baseline and proposed algorithm are then compared with ground truth, which are based on actual rendezvous. The experiments are performed in synthetic dataset, where we manually label the ground truth for actual rendezvous as described in Section 6.1. Accuracy and precision can be described as the function of **True Positive (TP)**, **True Negative (TN)**, **False Positive (FP)**, and **False Negative (FN)**. The confusion matrix is shown in Figure 18.

(3) Accuracy: Accuracy is the ratio of observation that are correctly predicted to the total number of observations, i.e., it provides how accurately the algorithms can predict the actual rendezvous, which may or may not take place. Based on Figure 18, the equation is as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}. \quad (12)$$

Trends: As we can see in Figure 19(a), the proposed TSS approach is more accurate than the baseline LTS approach with substantial improvement as we decrease the Effective Missing Period. This is due to the larger size of the MOBR in LTS as compared to MBR in TSS, which results in

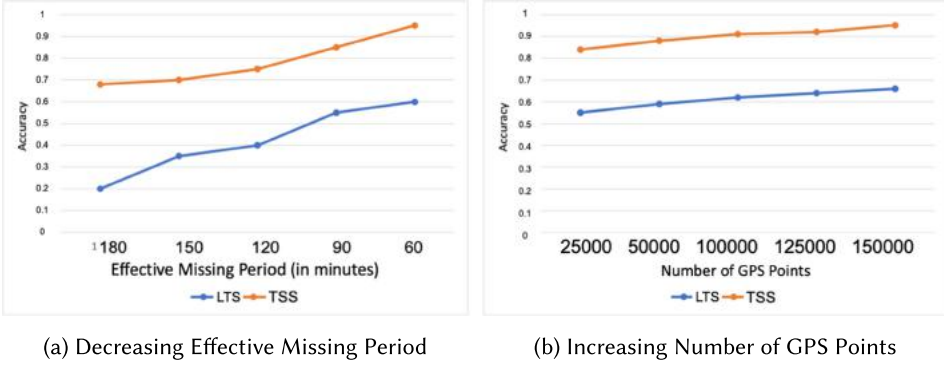


Fig. 19. Effect of varying parameters on accuracy.

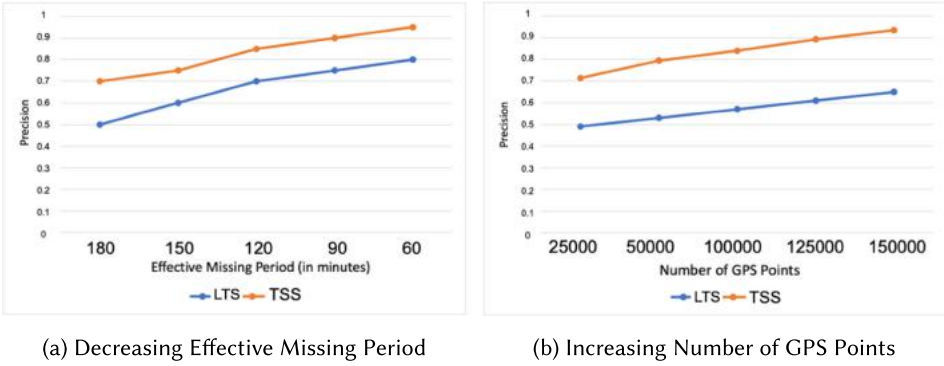


Fig. 20. Effect of varying parameters on precision.

high potential interaction among EMPs. This results in high number of potential candidate pairs thus resulting in high number of false positive results. Similarly, Figure 19(b) shows the proposed approach outperforms the baseline as we increase the number of GPS Points with fewer number of False Positives.

(4) Precision: Precision is the ratio of observations that are correctly predicted to the total number of predicted positive observations. This includes False Positives instances (i.e., when algorithm falsely predict the positive rendezvous). Hence, the lower is the number of false positives instances, the higher is the precision. Based on Figure 18, the equation is given below:

$$Precision = \frac{TP}{TP + FP}. \quad (13)$$

Trends: Similar trends are observed using the same parameters in Figure 20 showing that the proposed approach outperforms the baseline approach as we decrease the Effective Missing Period (Figure 20(a)) or increase the GPS points (Figure 20(b)). This is due to the fewer number of false positives generated in the proposed approach due to it is tighter minimum bounding rectangle as compared to the 3DR Tree used in the baseline.

7 RELATED WORK AND LIMITATIONS

An extensive amount of trajectory data is being generated via location-acquisition services and serves different research purposes such as anomaly detection, pattern recognition, and so on. The

survey in Reference [26] gives broad overview of trajectory data mining techniques related to pre-processing, data management, uncertainty, pattern mining, and so on. Spatiotemporal joins are the most common type of queries for trajectory query processing in real-world applications such as mobility analysis, rendezvous studies, and so on. In addition, there are trajectory similarity joins based on similarity functions such as Edit Distance [4], **Longest Common Sub-sequence (LCSS)** [16], and DTW [25]. A recent similarity join technique, TS-Join [21], uses spatial and temporal proximity via a two phase divide and conquer algorithm. Strain-Join [23] is a signature-based similarity join framework, which generates trajectory wise high quality signatures via a similarity metric and then filters out non similar trajectory pairs. Other techniques include a sliding window approach [1, 5] for enumerating candidate pairs and time interval threshold-based elimination [2, 7]. Clue-aware trajectory similarity [10] is used over trajectories with silent duration for clustering and aggregation based on behavioural patterns. Regarding query processing for spatiotemporal joins, there are methods based on 3D R-Trees such as Spatiotemporal R-Tree and TB-Tree [20].

Some frameworks [6, 19] employ reconstruction techniques for modeling uncertainty in gaps (or missing signals) in trajectory data. However, many of the techniques assume shortest path discovery or linear interpolation. These approaches are not designed to detect patterns when the trajectories of the moving objects are missing (e.g., due to weak signaling), since the objects may move far from the shortest path. More realistic solutions for modeling uncertainty in gaps are based on geometric and contextual models such as cylindrical [24] and space-time prism models [11, 17] that construct an areal interpolation of the gaps using coordinates and maximum speed of the objects. More recently, a kinetic prism model [14] showed improved estimation by considering other physical parameters such as uncertainty and acceleration. However, applying these models can be computationally expensive. Researchers in Reference [12] also presented the idea of intersecting two uncertainty regions using geometric properties of space-time prism. An alibi query proved to be an effective method for checking whether two space-time prisms intersect. However, their method is limited to theoretical simulations and not used in real-world applications such as analyzing generic and behavioral patterns [8], and so on. In this article, we leverage the geometric properties of space-time prism [12] for real-world applications and perform time slicing for better geometric approximation.

8 CONCLUSION AND FUTURE WORK

We extended our previous work on rendezvous detection in trajectory data with gaps. Our baseline algorithm is a Linear Temporal Scan [22] based on an approximating common minimum orthogonal bounding rectangle with a time-slicing operation for better area pruning effectiveness. We proposed a Temporal Selection Search algorithm, which leverages a time prioritizer to further reduce the time-slicing operation. In addition, we perform spatial optimization providing tighter bounds than baseline filters. The results show relatively better APE, computation time, accuracy, and precision for the proposed Temporal Selection Search as compared to the baseline approach.

Future Work: Computing approximate regions is very expensive in terms of time complexity and modeling them in regional space is challenging. We are exploring a grid indexing technique that is more efficient than linear search. Also, we plan to apply the proposed techniques in other application domains. We also plan to provide ground-truth verification for evaluation metrics on a real-world dataset. In addition, we plan to provide a detailed theoretical evaluation for the baseline and proposed algorithm. Finally, we plan to further refine the process of finding the exact geometry of a spatiotemporal intersection, since it is hard to find the exact geometry of the cone intersect due to its complexity.

APPENDICES

A TWO-DIMENSIONAL REPRESENTATION OF POSSIBLE RENDEZVOUS REGION

Given trajectory data with gaps, we find possible areas where moving objects (e.g., ships) rendezvous. Figure 21 shows a two-dimensional example of the problem Input and Output. Object 1 and Object 2's trajectory gaps lie between $p_3^{t=3}$ and $p_4^{t=4}$ with maximum speed 1 and 2, respectively. The output shows the candidate active volume (CAV) in the form of an ellipse for each object. A CAV is the region in a gap that represents all the possible locations of the object during a missing time interval. The intersection of the two CAVs and the circles generated from both segments is a spatiotemporal intersection (STI) that represents a possible rendezvous region at a certain time stamp for the two objects.

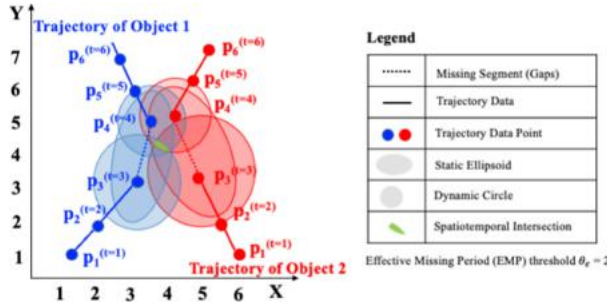


Fig. 21. Two-dimensional representation of a possible rendezvous region.

B ASYMPTOTIC TIME COMPLEXITY ANALYSIS FOR TIME-SLICING OPERATION

Time Slicing Operations: Both Linear Temporal Scan (LTS) and Temporal Selection Scan (TSS) perform time-slicing operations for better area pruning efficiency. When we evaluate each temporal slice $|T|$, both algorithms require rasterization of grid cells $grid(x, y)$ over the spatial filter approximated by the algorithm. If we perform a grid cell $grid(x, y)$ comparison over the spatial filter with the proposed TSS algorithm, then fewer grid cells need to be checked as compared to the baseline algorithm. The proposed approach also uses time prioritized-based temporal selection pruning, which narrows down the temporal range to when an actual intersection is likely to have occurred. This reduces the number of time-slicing operations and results in computational speedup as compared to the baseline algorithm.

ACKNOWLEDGMENTS

We thank Kim Koffolt and the spatial computing research group for their helpful comments and refinements.

REFERENCES

- [1] Petko Bakalov, Marios Hadjieleftheriou, Eamonn Keogh, and Vassilis J. Tsotras. 2005. Efficient trajectory joins using symbolic representations. In *Proceedings of the 6th International Conference on Mobile Data Management*. ACM, 86–93.
- [2] Petko Bakalov and Vassilis J. Tsotras. 2006. Continuous spatiotemporal trajectory joins. In *Proceedings of the International Conference on GeoSensor Networks*. Springer, 109–128.
- [3] BOEM and NOAA. 2020. MarineCadastr.gov. Retrieved from <https://marinecadastre.gov/ais/>.
- [4] Lei Chen, M. Tamer Özsu, and Vincent Oria. 2005. Robust and fast similarity search for moving object trajectories. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, 491–502.
- [5] Yun Chen and Jignesh M. Patel. 2009. Design and evaluation of trajectory join algorithms. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, 266–275.

- [6] Reynold Cheng, Jinchuan Chen, Mohamed Mokbel, and Chi-Yin Chow. 2008. Probabilistic verifiers: Evaluating constrained nearest-neighbor queries over uncertain data. In *Proceedings of the 24th International Conference on Data Engineering*. IEEE, 973–982.
- [7] Hui Ding, Goce Trajcevski, and Peter Scheuermann. 2008. Efficient similarity join of large sets of moving object trajectories. In *Proceedings of the 15th International Symposium on Temporal Representation and Reasoning*. IEEE, 79–87.
- [8] Somayeh Dodge, Robert Weibel, and Anna-Katharina Lautenschütz. 2008. Towards a taxonomy of movement patterns. *Info. Visual.* 7 (2008), 240–252.
- [9] Kathleen Hornsby and Max J. Egenhofer. 2002. Modeling moving objects over multiple granularities. *Ann. Artif. Intell.* 36 (2002), 177–194.
- [10] Chih-Chieh Hung, Wen-Chih Peng, and Wang-Chien Lee. 2015. Clustering and aggregating clues of trajectories for mining trajectory patterns and routes. *VLDB J.* 24 (2015), 169–192.
- [11] Hyun-Mi Kim and Mei-Po Kwan. 2003. Space-time accessibility measures: A geocomputational algorithm with a focus on the feasible opportunity set and possible activity duration. *J. Geogr. Syst.* 5 (2003), 71–91.
- [12] Bart Kuijpers, Rafael Grimson, and Walied Othman. 2011. An analytic solution to the alibi query in the space-time prisms model for moving object data. *Int. J. Geogr. Info. Sci.* 25 (2011), 293–322.
- [13] Bart Kuijpers, Harvey J. Miller, Tijs Neutens, and Walied Othman. 2010. Anchor uncertainty and space-time prisms on road networks. *Int. J. Geogr. Info. Sci.* 24 (2010), 1223–1248.
- [14] Bart Kuijpers, Harvey J. Miller, and Walied Othman. 2017. Kinetic prisms: Incorporating acceleration limits into space-time prisms. *Int. J. Geogr. Info. Sci.* 31 (2017), 2164–2194.
- [15] Mei-Po Kwan. 2004. GIS methods in time-geographic research: Geocomputation and geovisualization of human activity patterns. *Geografiska Annaler: Ser. B, Hum. Geogr.* 86 (2004), 267–280.
- [16] Rake Agrawal King-lp Lin and Harpreet S. Sawhney Kyuseok Shim. 1995. Fast similarity search in the presence of noise, scaling, and translation in time-series databases. In *Proceeding of the 21st International Conference on Very Large Data Bases*. Citeseer, 490–501.
- [17] Harvey J. Miller. 1991. Modelling accessibility using space-time prism concepts within geographical information systems. *Int. J. Geogr. Info. Sci.* 5 (1991), 287–301.
- [18] Tijs Neutens, Tim Schwanen, and Frank Witlox. 2011. The prism of everyday life: Towards a new research agenda for time geography. *Transport. Rev.* 31 (2011), 25–47.
- [19] Dieter Pfoser and Christian S. Jensen. 1999. Capturing the uncertainty of moving-object representations. In *Proceedings of the International Symposium on Spatial Databases*. Springer, 111–131.
- [20] Dieter Pfoser, Christian S. Jensen, Yannis Theodoridis, et al. 2000. Novel approaches to the indexing of moving object trajectories. In *Proceedings of the 26th International Conference on Very Large Databases (VLDB'00)*. VLDB Endowment, 395–406.
- [21] Shuo Shang, Lisi Chen, Zhewei Wei, Christian S. Jensen, Kai Zheng, and Panos Kalnis. 2017. Trajectory similarity join in spatial networks. In *Proceedings of the International Conference on Very Large Databases (VLDB'17)*. VLDB Endowment.
- [22] Arun Sharma, Xun Tang, Jayant Gupta, Majid Farhadloo, and Shashi Shekhar. 2020. Analyzing trajectory gaps for possible rendezvous: A summary of results. In *Proceedings of the 11th International Conference on Geographic Information Science (GIScience'21)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [23] Na Ta, Guoliang Li, Yongqing Xie, Changqi Li, Shuang Hao, and Jianhua Feng. 2017. Signature-based trajectory similarity join. *IEEE Trans. Knowl. Data Eng.* 29 (2017), 870–883.
- [24] Goce Trajcevski, Alok Choudhary, Ouri Wolfson, Li Ye, and Gang Li. 2010. Uncertain range queries for necklaces. In *Proceedings of the 11th International Conference on Mobile Data Management*. IEEE, 199–208.
- [25] Byoung-Kee Yi, Hosagrahar V. Jagadish, and Christos Faloutsos. 1998. Efficient retrieval of similar time sequences under time warping. In *Proceedings of the 14th International Conference on Data Engineering*. IEEE, 201–208.
- [26] Yu Zheng. 2015. Trajectory data mining: An overview. *ACM Trans. Intell. Syst. Technol.* 6 (2015), 1–41.

Received December 2020; revised March 2021; accepted May 2021